
Parte 1: Teoría: Arquitecturas y las tecnologías de programación sobre clientes web (1 punto)

El presente bloque se compone de preguntas de opción múltiple con respuesta única. Para cada ítem, usted deberá seleccionar la opción que considere correcta. Cada pregunta vale 0,10 puntos. Para aplicar el factor de corrección por azar y asegurar la rigurosidad de la prueba, se establece un sistema de penalización: cada dos (2) respuestas incorrectas anularán el valor de una (1) respuesta correcta. Las preguntas que sean dejadas en blanco no sumarán ni restarán puntos.

1.- ¿En qué escenario muy específico un JavaScript interno (embebido), a pesar de las desventajas de mantenimiento, ofrece un ligero beneficio de rendimiento sobre un archivo externo?

- A.- Cuando se carga junto con la librería React o Angular.
- B.- Cuando el script utiliza solo funciones puras y no modifica variables globales.
- C.- Cuando el código es extremadamente corto y la latencia de la conexión es muy alta, ya que se elimina la necesidad de una petición HTTP adicional.**
- D.- Cuando el script interactúa con más de 10 elementos del DOM simultáneamente.

2.- Si un desarrollador necesita cargar un script de gran tamaño sin que bloquee el parseo inicial del DOM, ¿cuál es el atributo que se debe usar en la etiqueta <script src="..."> del fichero externo para que este se descargue en paralelo y se ejecute solo después de que el HTML haya sido completamente parseado, respetando el orden de los scripts?

- A.- async
- B.- inline
- C.- defer**
- D.- sinc

3.- ¿Cuál es una ventaja principal de usar archivos .js externos en lugar de código embebido?

- A.- Oculta completamente el código fuente del usuario.
- B.- Permite reutilizar el mismo código en múltiples páginas.**
- C.- Acelera la ejecución del código en el navegador.
- D.- Elimina la necesidad de validar la sintaxis.

4.- ¿Para qué sirve la etiqueta <noscript> en una página web?

- A.- Para desactivar temporalmente el JavaScript en la página.
- B.- Para mostrar contenido alternativo cuando el navegador no soporta JavaScript.**
- C.- Para incluir scripts que solo funcionan en servidor.
- D.- Para optimizar el rendimiento de los scripts.

5.-Cuál de las siguientes NO es una característica de JavaScript?

- A.- Lenguaje interpretado.
- B.- Tipado dinámico.
- C.- Ejecutado de forma nativa dentro del navegador.
- D.- Necesita compilarse manualmente antes de usarse en la web.**

DWEC - 1ª EVALUACION - 2025-2026

- 6.- ¿Qué problema surge al validar un formulario SOLO en cliente?
- A.- Los usuarios con JavaScript desactivado no pueden enviar el formulario.
 - B.- Los datos pueden ser manipulados antes de llegar al servidor.**
 - C.- Los mensajes de error no se muestran inmediatamente.
 - D.- Se requiere jQuery obligatoriamente.
- 7.- ¿por qué se considera JavaScript un lenguaje interpretado?
- A.- Porque el navegador ejecuta directamente el código sin necesidad de una fase de compilación previa explícita por parte del programador.**
 - B.- Porque siempre se convierte antes a código máquina mediante un compilador externo.
 - C.- Porque requiere generar archivos binarios .exe antes de poder ejecutarse.
 - D.- Porque solo puede ejecutarse en el servidor y no en el navegador.
- 8.- ¿Qué política de seguridad restringe que scripts de un dominio accedan a recursos de otro dominio diferente?
- A.- Política de cookies
 - B.- Política del mismo origen (Same-Origin Policy)**
 - C.- Política de caché del navegador
 - D.- Política de ejecución segura
- 9.- ¿Cuál NO es una característica típica de los editores de texto para programación web mencionados?
- A.- Sintaxis de colores para diferentes lenguajes.
 - B.- Verificación de sintaxis en tiempo real.
 - C.- Generación de partes de código automáticas.
 - D.- Compilación automática a código máquina.**
- 10.- ¿Qué se puede hacer en la consola de DevTools además de ver mensajes de log?
- A.- Solo ver errores y advertencias.
 - B.- Modificar archivos en el servidor remotamente.
 - C.- Ejecutar código JavaScript en el contexto de la página actual.**
 - D.- Reiniciar el servidor web.

Parte 2: Creación y Manipulación Estándar de la Lista (2 puntos)

El primer objetivo es gestionar una lista de países. Para ello, debe comenzar inicializando el sistema declarando una constante llamada PAISES y asignarle un array que contenga al menos seis nombres de países (Ejemplo: `["España", "Francia", "Italia", "Alemania", "Portugal", "Malta"]`)

A partir de esta lista inicial, implemente una serie de funciones de gestión:

- Implemente una función **contarPaíses** que nos permita conocer rápidamente la cantidad total de elementos en la lista, devolviendo dicho número. (0,25 pts)

DWEC - 1ª EVALUACION - 2025-2026

- Cree una función **inversa** que devuelva un nuevo array con la lista de países en orden inverso al original, asegurándose de que el array PAISES no sea modificado. (0,25 pts)
- Necesitamos una función **agregarPais** que reciba un nuevo país como parámetro y lo añada al final de la lista; esta función debe retornar la nueva longitud total del array tras la adición. (0,25 pts)
- Desarrolle una función **bajaPais** que nos permita eliminar el primer país de la lista por motivos de baja, y que nos devuelva el nombre del país que ha sido eliminado. (0,25 pts)

A continuación, utilice los métodos funcionales de JavaScript para transformar y seleccionar datos sin modificar el original:

- Cree la función **formatearPaises**(array). Encargada de devolver un nuevo array donde cada nombre de país se haya transformado en un objeto que contenga dos propiedades: nombre (el nombre original del país) y longitud (el número de caracteres de ese nombre). El retorno esperado es un array de objetos, como por ejemplo: [{nombre: "España", longitud: 6}, {nombre: "Francia", longitud: 7}, ...]. (0,5 pts)
- Cree la función **filtrarPaises**(array). Encargada de devolver un nuevo array que solo contenga aquellos países cuyo nombre comience por una vocal ('A', 'E', 'I', 'O', 'U'), sin distinguir entre mayúsculas y minúsculas. El retorno esperado es un array de cadenas de texto, como por ejemplo: ["España", "Italia", ...]. (0,5 pts)

Parte 3: Contenido Bloqueado (1,5 puntos)

Cree una función llamada **bloquearContenido** que debe recorrer todo el Árbol del Documento (DOM) desde la etiqueta <body>.

- Si la función encuentra cualquier elemento de texto que contenga la palabra "sexo" (o alguna palabra clave sensible elegida por el profesor), deberá eliminar ese elemento y sustituirlo por el texto "Contenido Bloqueado" escrito en negrita. (1 pto)
- En caso de que encuentre la palabra "Examen", deberá emitir una alerta con el mensaje "¡DETECTADO!" y, además, escribir en la consola del navegador el elemento contenedor (node) de la palabra "Examen". (0,5 pts)
- NOTA: Toda la estructura de la página debe generarse desde JavaScript utilizando métodos como document.createElement() y appendChild().

Parte 4: Gestión de Formulario y Eventos (4 puntos)

A partir de la tabla HTML proporcionada (con su respectiva caja de texto y botón de Añadir):

```
<input type="text" id="texto" />
<button onclick="agregarFila()">Añadir fila</button>
<br />
<table>
  <thead>
    <tr><th>Contenido</th><th>Operación</th></tr>
  </thead>
  <tbody id="bodyTabla">
    <tr>
      <td id="fila1">Cecilia me ha dicho que este es el texto a canificar</td>
      <td><button onclick="toCani('fila1')">Caniar</button></td>
    </tr>
  </tbody>
</table>
```

Usted deberá añadir el código JavaScript necesario para implementar las siguientes funcionalidades:

- **Funcionalidad del Botón "Añadir fila":** Implemente la función `agregarFila` de modo que, al pulsarse, tome el contenido del campo de texto y lo añada como una nueva fila a la tabla, incluyendo su respectivo botón de "**Caniar**" que apunte a la nueva celda. Toda la estructura de la página debe generarse desde JavaScript utilizando métodos como `document.createElement()` y `appendChild()`. (1 pto)
- **Evento Mouseover:** Implemente la lógica para que, al pasar el ratón por encima de cualquier celda de la tabla, el color de fondo de esa celda cambie (por ejemplo, a gris claro), volviendo a la normalidad al retirar el ratón. (1 pto)
- **Función de Transformación:** Cree la función `toCani(cadena)`. Esta función debe recibir una cadena de texto y devolverla transformada bajo las siguientes reglas "Cani": (2 ptos)
 - Alternancia: Alternar el uso de MAYÚSCULAS y minúsculas.
 - Sustituciones fonéticas: Sustituir la letra C por la K. Se valorará el uso de expresiones regulares para sustituir las secuencias "ca", "co", "cu" por "ka", "ko", "ku" (pero no "ce", "ci").
 - Sustituciones compuestas: Sustituir todas las ocurrencias de "qu" por "k" y todas las "ch" por "x".
 - Añadido final: Añadir tres letras H al final de la cadena resultante.
 - Por ejemplo, si le pasamos a la función la cadena "una cadena cani es como esta" obtendremos "UnA KaDeNa kAnI Es kOmO EsTaHHH".

Parte 5: Modelado de Objetos (1,5 puntos)

Define em Javascript un tipo de objeto **Cometa** cuyas propiedades de instancia (específicas de cada objeto) sean **diametro**, **temperatura** y **nombre**. La temperatura será un valor numérico que suponemos está en grados centígrados.

- Como “propiedad común” a todos los objetos de tipo cometa define:
 - **definicionSegunDiccionario** (que debe contener la definición de cometa según el diccionario)
- Como métodos comunes :
 - **obtenerRadio** (que debe devolver el radio)
 - **obtenerTemperaturaFahrenheit** (que debe devolver el valor de temperatura expresado en grados Fahrenheit cuya formula: $(temperatura * (9/5)) + 32$).

Crea tres objetos de tipo cometa y comprueba que puedes acceder tanto a las propiedades específicas como a las propiedades comunes y métodos comunes desde cada objeto.

CRITERIOS

- Se tendrá en cuenta el uso de sintaxis avanzada de JavaScript, eficiencia y código bien estructurado, comentarios y diseño adecuado de interfaz. Se penalizará negativamente su falta de uso (hasta 2 puntos menos)
- Se valorará positivamente no usar etiquetas HTML en el archivo fuente.
- Toda la estructura de la página debe generarse desde JavaScript utilizando métodos como document.createElement() y appendChild().
- Se valorará el uso de funciones auxiliares re-utilizables para generar elementos HTML y gestionar eventos.
- Se valorará la eficiencia del código
- Se valorará el uso buenas prácticas de programación y sintaxis avanzada de JavaScript.
- Se revisará el correcto manejo de eventos como clicks, ordenación, hover y actualizaciones dinámicas